



Final Report

Skill Development Project I - ICT 1108
Bachelor of Information and Communication
Technology (BICT)
Degree Programme

Department of Information and Communication
Technology
Faculty of Technology
Rajarata University of Sri Lanka
Mihintale



Details of the Project

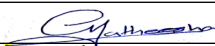
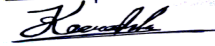
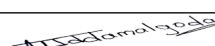
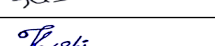
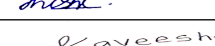
Project Title : Smart Finance Managing Application for University Students

Group Number : Group 11

Group Name : Apex Developers

Submission Date : 2026.09.18

Group Members :

Student Name	Index Number	Signature
R.K.M.Chathuranga	2709	
D.M.K.N.Disanayake	2716	
H.S.M.Edman	2720	
W.D.R.A.U.Iddamalgoda	2731	
W.A.K.Kalpani	2738	
P.T.S.Karunanayake	2733	

Internal Supervisor(s) :

Name : Dr.P.R.H.N.G. Thilakarathne

Designation : Senior Lecturer

Department : ICT

Email : _____

Signature : _____ Date: _____

External Supervisor(s) :

Name : _____

Designation : _____

Email : _____

Phone Number : _____

Signature : _____ Date: _____



Table of Contents

1. Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Aim	1
1.4 Objectives	1
2. Approach	2
2.1 Scope of the Project	2
2.2 End Users of the Project	2
2.3 Functional Requirements	2
2.4 Non-Functional Requirements	2
3. Methodology / Technical Approach	3
3.1 System Design	3
3.2 Architecture Diagram	3
3.3 User Interface Design	3
3.4 Methods and Technologies Used	6
4. Development	6
4.1 Solution	6
4.2 Codes	6
5. Conclusion	7
5.1 Conclusions of the Project	7
5.2 Lessons Learned and Skills Earned	7
5.3 Recommendations for Improvements	7
6. References	8



Table of Figures

Figure 1 Three-Tier System Architecture Diagram.....	3
Figure 2 Login Screen.....	4
Figure 3 Main Dashboard.....	4
Figure 5 Add Transaction.....	4
Figure 4 Voice Entry Screen.....	4
Figure 7 Setting & Update Monthly Allowance.....	5
Figure 6 History Screen.....	5
Figure 9 Categories screen.....	5
Figure 8 Add Transaction.....	5

1. Introduction

1.1 Introduction

University students regularly encounter financial distress when managing monthly allowances, primarily driven by the continuous occurrence of unrecorded daily micro-transactions—such as snacks, tea, or local transport fares. While minor on an individual basis, these expenses accumulate significantly over a monthly cycle, causing students to exhaust their funds prematurely. Managing tight monthly allowances requires consistent record-keeping, yet traditional manual financial logging methods introduce prohibitive friction into daily routines.

1.2 Problem Statement

Existing personal finance applications fail to accommodate the routine behavioral needs of university students due to two primary issues:

- **High Friction in Expense Logging:** Standard applications rely on manual data entry, which is time-consuming on the move. Statistics show that roughly 69.5% of students abandon expense tracking due to entry effort, while 73.9% explicitly dislike manual typing [1].
- **Inflexible Budgeting Models:** Traditional applications evaluate spending against a static, flat monthly total. They fail to separate recurring fixed obligations (e.g., accommodation fees, tuition, utilities) from daily flexible spending, preventing students from gauging their actual safe daily spending capacity.

1.3 Aim

To design and develop a lightweight, voice-driven mobile application that eliminates micro-transaction logging friction and dynamically calculates a daily safe spending limit for university students.

1.4 Objectives

- Implement a speech recognition module enabling users to log micro-transactions instantly via voice input.
- Develop an automated budgeting algorithm that deducts fixed monthly obligations and computes a dynamic safe daily allowance:
- Design an intuitive visual dashboard equipped with graphical expense breakdowns and automated threshold alerts.
- Provide an instantaneous manual fallback editing mechanism for quiet or noisy environments.

2. Approach

2.1 Scope of the Project

- **In-Scope:** Voice-to-text transaction logging, dynamic daily budget calculation, manual fallback input, visual analytics dashboard, local threshold alert notifications, and user authentication.
- **Out-of-Scope:** Direct bank API integrations, optical character recognition (OCR) receipt scanning, automated SMS parsing, and offline local storage mechanisms.

2.2 End Users of the Project

The primary end users are undergraduate and postgraduate university students managing fixed monthly allowances or casual incomes who require low-friction tools to track daily micro-transactions.

2.3 Functional Requirements

- **Voice-to-Text Entry:** System must convert speech input into structured numerical amounts and expense categories.
- **Dynamic Budgeting:** System must re-evaluate the safe daily limit automatically upon every expense submission or budget modification.
- **Manual Entry & Correction:** System must allow manual transaction inputs and immediate post-voice editing.
- **Alert Notifications:** System must trigger local push notifications when daily spending approaches or exceeds calculated limits.
- **Historical Analysis:** System must display a filterable, chronological list of past transactions.

2.4 Non-Functional Requirements

- **Performance & Usability:** Voice expense logging completion time must be under 5 seconds.
- **Cross-Platform Support:** UI must operate consistently across iOS and Android mobile platforms.
- **Security:** User session management and credentials must be secured using JSON Web Tokens (JWT) [2].
- **Reliability:** Speech recognition fallback must maintain 100% availability via manual input option during API latency or background noise failures.



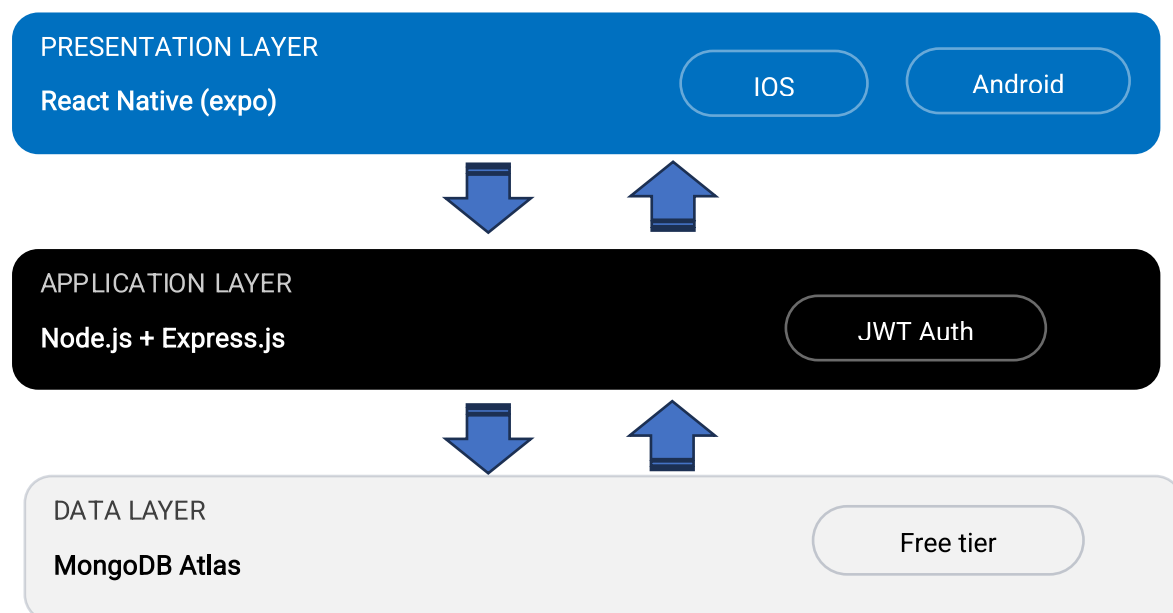
3. Methodology / Technical Approach

3.1 System Design

The system adopts a standard Three-Tier Architecture [3]:

- **Presentation Layer:** Mobile application frontend rendering user interfaces and capturing voice/text inputs.
- **Application Layer:** RESTful API backend executing business logic, budget processing, and user authorization.
- **Data Layer:** Cloud NoSQL database maintaining user profiles, fixed settings, and transaction records.

3.2 Architecture Diagram



3.3 User Interface Design

The frontend consists of six primary screens:

- **Splash Screen:** Handles session state initialization.
- **Login / Signup Screen:** Manages email/password authentication.
- **Main Dashboard:** Displays the dynamic Safe Daily Limit, total monthly balance, visual charts, and a central Floating Microphone Button.
- **Manual Entry :** Form input fallback for manual entries.
- **Voice Entry :** Captures voice commands, parses speech into numerical amounts and categories, displays match accuracy confidence, and provides

quick confirmation controls.

- **History Screen:** Chronological, filterable transaction list.
- **Settings Screen:** Configurations for allowance, fixed expenses, and alert parameters.

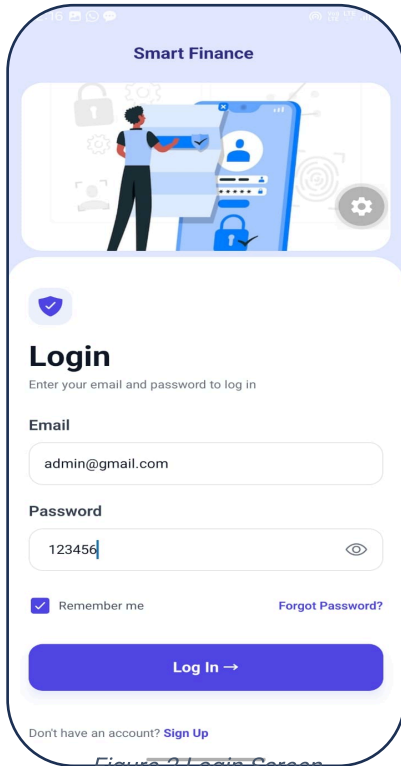


Figure 2 Login Screen

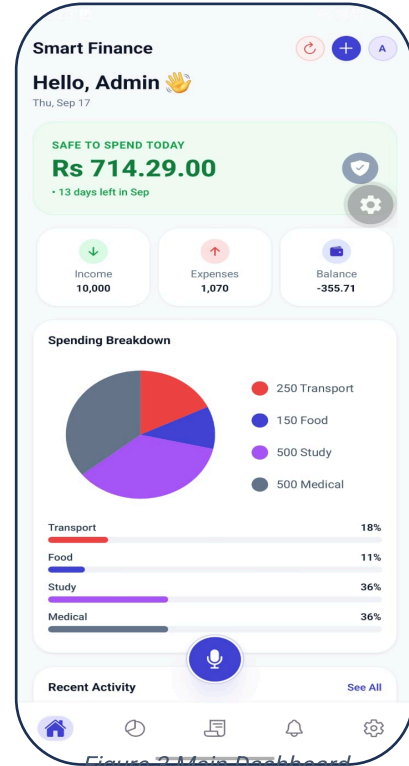


Figure 3 Main Dashboard

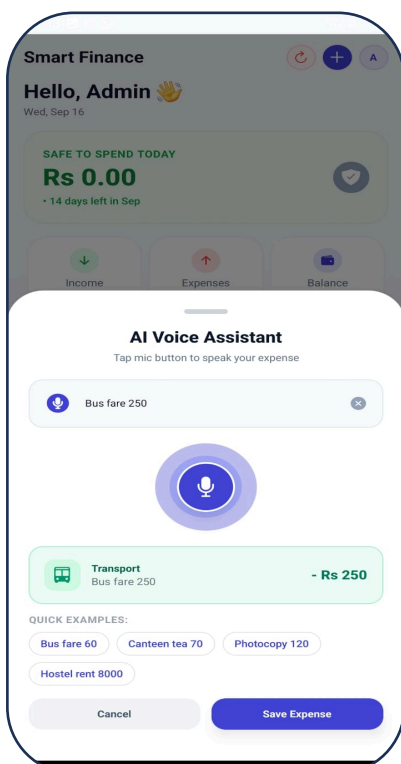


Figure 4 Voice Entry Screen

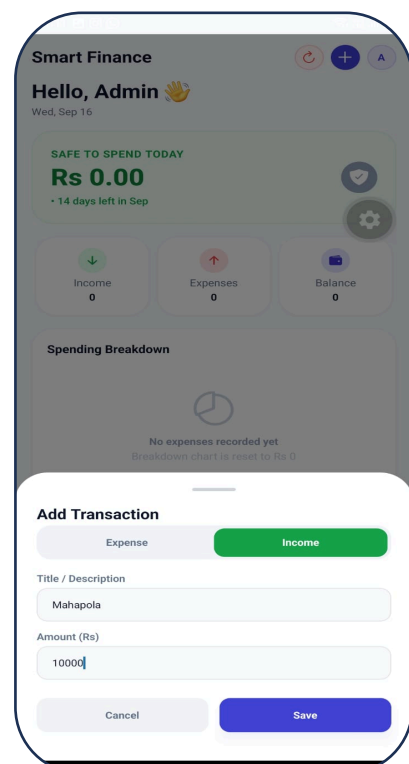


Figure 5 Add Transaction

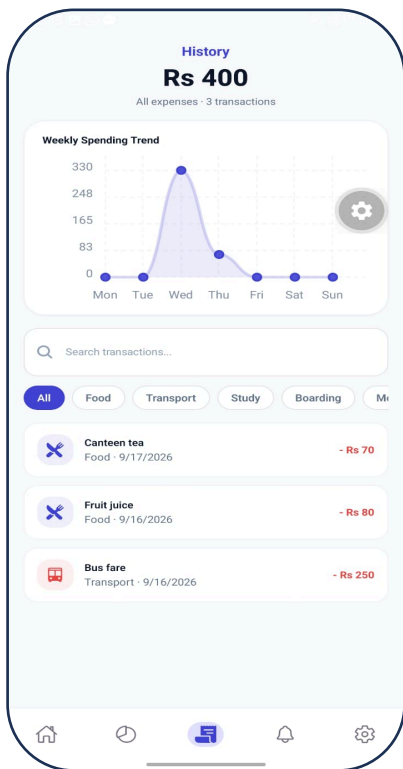


Figure 6 History Screen

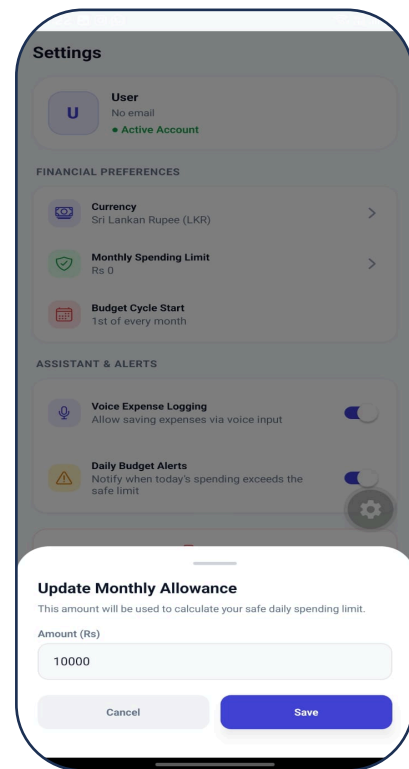


Figure 7 Setting & Update Monthly Allowance

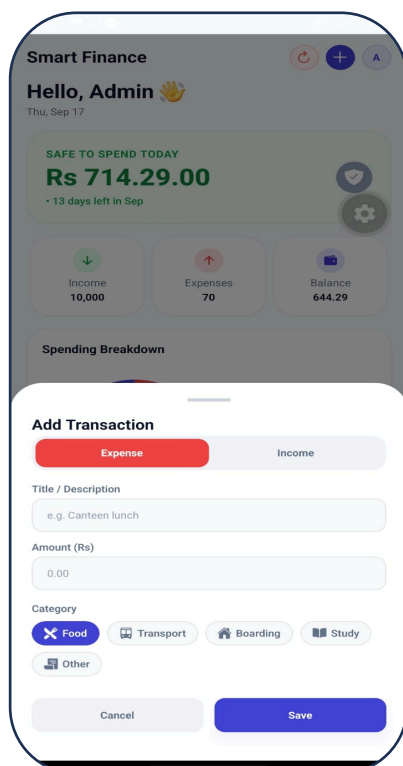


Figure 8 Add Transaction

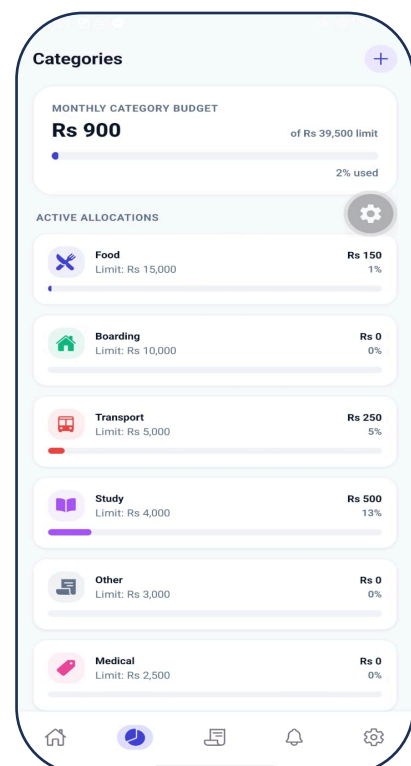


Figure 9 Categories screen

3.4 Methods and Technologies Used

- **Frontend:** React Native [4] with Expo [5] (Cross-platform framework).
- **Backend:** Node.js [6] with Express.js [7] framework (RESTful services).
- **Database:** MongoDB Atlas [8] with Mongoose ODM [9] (Cloud NoSQL document store).
- **Key Libraries:** Expo Audio (Speech recognition), React Native Chart Kit (Visual analytics), Expo Notifications (Push alerts), and JSON Web Token (Security authentication) and Postman [10].

4. Development

4.1 Solution

The developed Minimum Viable Product (MVP) implements voice-driven expense capturing paired with real-time budget calculations. User speech is processed, categorized, and converted into structured database records. The Safe Daily Limit updates instantly on the client dashboard upon data persistence. Usability tests confirmed that voice logging significantly reduced entry duration compared to manual typing forms.

4.2 Codes

We implemented **authentication.js** as a dedicated authentication middleware to verify the issued JSON Web Token (JWT) upon user login, securely binding the authenticated user's identity to **req.user**.

Leveraging a modular MVC architecture alongside **async/await** execution patterns, the backend safely retrieves financial data—such as expenses and allowance settings—belonging exclusively to the logged-in user, thereby enforcing strict user data isolation.

Throughout the authentication lifecycle, all endpoint interactions yield clear JSON responses paired with standard HTTP status codes (200 Success, 401 Unauthorized, 500 Server Error), ensuring the security and data reliability of the frontend mobile application.

```
import jwt from 'jsonwebtoken';

export default function authenticateUser(req, res, next) {
  // Extract Authorization header
  const header = req.header("Authorization");
```

```

// Return 401 if header is missing
if (!header) {
  return res.status(401).json({
    message: "Access denied. No token provided. Please log in or register."
  });
}

// Extract Bearer token
const token = header.replace("Bearer ", "");

// Verify JWT token
jwt.verify(token, process.env.JWT_SECRET, (error, decoded) => {
  if (error || !decoded) {
    return res.status(401).json({
      message: "Invalid token. Please log in again."
    });
  }

  // Attach user data to req.user for data isolation
  req.user = decoded;
  next();
});
}

```

5. Conclusion

5.1 Conclusions of the Project

The Smart Finance Managing Application addresses the behavioral barriers of personal finance management for university students. Replacing tedious manual entry with voice recognition and shifting from flat monthly budgets to dynamic daily spending limits provides students with an actionable tool to maintain discipline over daily micro-transactions.

5.2 Lessons Learned and Skills Earned

- **Speech Recognition Edge Cases:** Realized the necessity of manual fallback integration due to ambient noise interference in public environments.
- **Full-Stack Mobile Architecture:** Gained expertise in connecting React Native frontends with Node.js/Express backends and MongoDB cloud databases.
- **User-Centric UI Engineering:** Learned to optimize UI patterns specifically around reducing user input friction.

5.3 Recommendations for Improvements

- **Offline Storage Integration:** Integrate local storage solutions (e.g., SQLite or WatermelonDB) for offline transaction recording.
- **Automated SMS Parsing:** Implement background transaction detection from

mobile banking alerts.

- **OCR Receipt Capture:** Incorporate optical character recognition to automatically extract itemized cost data from paper receipts.



References

- [1] A. D. (. 11), "Smart Finance Manager Student Survey," Google Forms, 20 July 2026. [Online]. Available: https://docs.google.com/spreadsheets/d/1fJpZcpwxhjO4wPjEo-tA6XWF3q9qkDivHM12_UPAQs/edit?usp=sharing. [Accessed: Sep. 18, 2026]..
- [2] J. B. a. N. S. M. Jones, "JSON Web Token (JWT)," IETF RFC 7519," 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [3] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," California, Irvine, CA, USA, 2000.
- [4] M. O. Source, "React Native Documentation," 2024. [Online]. Available: <https://reactnative.dev/>.
- [5] Expo, "Expo Audio and Framework API Documentation," 2024. [Online]. Available: <https://docs.expo.dev/>.
- [6] N. Foundation, "Node.js v20 API Documentation," 2024. [Online]. Available: <https://nodejs.org/docs/>
- [7] Express.js, "Express - Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>.
- [8] M. Inc., "MongoDB Atlas Cloud Database," 2024. [Online]. Available: <https://www.mongodb.com/atlas>.
- [9] M. ODM, "Mongoose Documentation v8.0.0," 2024. [Online]. Available: <https://mongoosejs.com/docs/>.
- [10] Postman, "Postman API Platform," 2024. [Online]. Available: <https://www.postman.com/>.

